Lab 4 Report

Chen Liangkun 23020036007. Lab group: 4.

Lab date: 24th November 2025. Report date: 4th December 2025.

1 Lab Introduction and Team Information

In this experiment, we will integrate the core components learned in the previous experiments and apply Trajectory Optimization techniques to finally get your simulated quadcopter to complete an exciting task: Drone Racing..

2 Individual Work

2.1 Single-segment trajectory optimization

Consider the following minimum velocity single-segment trajectory optimization problem:

$$\min_{P(t)} \quad \int_0^1 (P^{(1)}(t))^2 dt, \tag{1}$$

$$s.t. \quad P(0) = 0, \tag{2}$$

$$P(1) = 1, \tag{3}$$

$$P(t) = p_N t^N + p_{N-1} t^{N-1} + \cdots + p_1 t + p_0. \tag{4}$$

1. Suppose we restrict $P(t) = p_1 t$ to be a polynomial of degree 1, what is the optimal solution of problem (1)? What is the value of the cost function at the optimal solution?

   Ans: Notice $p_0 = 0$ and the constraint $P(1) = 1$. So the P(t) can we can totally define P(t).

   $$P(t) = 1 \times t + 0. \tag{5}$$

   Therefore its optimal solution is also the t . Now we calculate the cost function

   $$J = \int_0^1 (\dot{P}(t))^2 dt = 1. \tag{6}$$

2. Suppose now we allow P(t) to have degree 2, i.e. $P(t) = p_2 t^2 + p_1 t$,

(a) Write $\int_0^1 (P^{(1)}(t))^2 dt$, the cost function of problem (1), as $\boldsymbol{p}^T \boldsymbol{Q} \boldsymbol{p}$, where $\boldsymbol{p} = [p_1, p_2]^T$ and $\boldsymbol{Q} \in \mathcal{S}^2$ is a symmetric 2×2 matrix.

Answer:Consider that

$$J = \int_0^1 (p_2^2 + p_1 t)^{(1)^2} dt \tag{7}$$

$$\frac{4}{3} p_2^2 + 2p_1 p_2 + p_1^2, \tag{8}$$

$$\mathbf{p^T Q p} = \begin{bmatrix} p1 & p2 \end{bmatrix} \begin{pmatrix} 1 & 1 \\ 1 & \frac{4}{3} \end{pmatrix} \begin{bmatrix} p1 \\ p2 \end{bmatrix} \tag{9}$$

(b) Write $P(1) = 1$, constraint (3), as $\boldsymbol{A}\boldsymbol{p} = \boldsymbol{b}$, where $\boldsymbol{A} \in R^{1\times 2}$ and $\boldsymbol{b} \in R$

Ans: We can see that

$$p1 + p2 = 1 \quad \textbf{constraint(3)}, \tag{10}$$

$$\mathbf{Ap} = \mathbf{b} \longrightarrow \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} p1 \\ p2 \end{bmatrix} = 1 \tag{11}$$

where $\mathbf{A} = \begin{bmatrix} 1 & 1 \end{bmatrix}$ and $b = 1$.

(c) Solve the Quadratic Program (QP):

$$\min_{\boldsymbol{p}} \boldsymbol{p}^T \boldsymbol{Q} \boldsymbol{p} \quad s.t. \quad \boldsymbol{A}\boldsymbol{p} = \boldsymbol{b}. \tag{12}$$

Ans:

```
 H = [2 2;
    2 2];
f = [0;0];
Aeq = [1, 1];
beq = 1;
A =[];
b =[];
p = quadprog(H,f,A,b,Aeq,beq)

p =

1.0000
0.0000
```

Where $J^* = 1$, it is same as the degree 1. So we are unable to get lower value.

3. Now suppose we allow $P(t) = p_3 t^3 + p_2 t^2 + p_1 t$

(a) Let $\boldsymbol{p} = [p_1, p_2, p_3]^T$, write down $\boldsymbol{Q} \in \mathcal{S}^3, \boldsymbol{A} \in R^{1\times 3}, \boldsymbol{b} \in R$ for the QP (5).

Ans: We consider

$$\int_0^1 (9p_3^2 t^4 + 12p_2 p_3 t^3 + (4p_2^2 + 6p_1 p_3)t^2 + 4p_1 p_2 t + p_1^2) dt, \tag{13}$$

After some algebraic calculation

$$\mathbf{p^T Q p} = \mathbf{p^T} \begin{pmatrix} 1 & 1 & 1 \\ 1 & \frac{1}{2} & \frac{3}{2} \\ 1 & \frac{3}{2} & \frac{3}{5} \end{pmatrix} p, \tag{14}$$

$$\mathbf{Ap} = \mathbf{b} \rightarrow \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \mathbf{p} = 1 \quad constraint3 \tag{15}$$

(b) Solve the QP, what optimal solution do you get? Do this example agree with the result we learned from Euler-Lagrange equation in class?

Ans:

```
>> H = [2 2 2;2 8/3 3;2 3 18/5]

H =

    2.0000    2.0000    2.0000
    2.0000    2.6667    3.0000
    2.0000    3.0000    3.6000

>> f = [0;0;0]

f =

     0
     0
     0

>> A=[];b=[];
>> Aeq = [1 1 1]

Aeq =

     1     1     1

>> beq = 1

beq =

     1

>> p = quadprog(H,f,A,b,Aeq,beq)

p =

1.0000
0.0000
-0.0000
```

We can gather two basic fact in the solution: the minimum cost is always 1. Yess, Euler-Lagrange equation result (general case): The optimal solution

for minimizing the r order derivative ($\mathcal{L} = (P^{(r)})^2$) is a $N = 2r - 1$ order polynomial. Example $r = 1$ (minimum speed): $N = 2(1) - 1 = 1$ order polynomial.

4. Now suppose we are interested in adding one more constraint

$$\min_{P(t)} \int_0^1 (P^{(1)}(t))^2 dt, \ s.t. \quad P(0) = 0, \qquad P(1) = 1, \qquad P^{(1)}(1) = -2. \tag{16}$$

Using the QP method above, find the optimal solution and optimal cost of problem ( ) in the case of:

(a) $P(t) = p_2 t^2 + p_1 t$

(b) $P(t) = p_3 t^3 + p_2 t^2 + p_1 t$

Ans: As the new constraint shows

$$2p_2 + p_1 = -2 \tag{17}$$

So we can get $p_1 = 4 \quad p2 = -3$ in a. Therefore the optimal cost of the problem is defined which is $J^* = -3t^2 + 4t \quad J = 4$.

For b there is more complex but we still can use great matlab

```
Aeq(2,:) = [3 2 1]

Aeq =

     1     1     1
     3     2     1

>> beq(2,:) = 2

beq =

     1
    -2

>> p = quadprog(H,f,A,b,Aeq,beq)

p =

 2.2500
-7.5000
 6.2500
```

Which the J is still equal to 1.

2.2 Multi-segment trajectory optimization

1. Assume our goal is to compute the minimum snap trajectory over segments. How many and which type of constraints (at the intermediate points and at the start and end of the trajectory) do we need in order to solve this problem? Specify the number of waypoint constraints, free derivative constraints and fixed derivative constraints.

   Ans: Through Euler-Lagrange equation as this is a snap optimization the r is equal to 4, so N = 2(4) - 1 which is a 7th polynomial. And we have $8k$ unknown parameter. We need to construct $8k$ linear constraint equations to uniquely determine these parameters.

   (1) Waypoint Constraints: These constraints ensure that the starting and ending points of each trajectory segment are at the correct location. Logic: Each segment has a start position and an end position. Segment 1: The starting point is Start and the ending point is Waypoint_1. Segment i: The starting point is Waypoint_{i-1} and the end point is Waypoint_i. Quantity: Each segment provides 2 position constraints (start $x(t_{start})$ and end $x(t_{end})$). Total: $2k$ constraints.

   (2) Free Derivative Constraints (Continuity) These constraints ensure that the trajectory is smooth and continuous at intermediate waypoints (connections). Number of intermediate points: For k segments, there are $k-1$ intermediate points. Continuity requirement: According to the handout (refer to $r = 2$ for the graphical logic), to ensure smoothness, we usually force continuity up to $N-1$ order derivatives if no additional restrictions are imposed. Here $N = 7$, so we need to ensure that the 1st to 6th order derivatives are continuous at the connection point. That is: $\dot{x}, \ddot{x}, \dddot{x}, x^{(4)}, x^{(5)}, x^{(6)}$ are equal to the left and right of the connection. Quantity: $6 continuity constraints per intermediate point. Total$:6(k-1)$constraints$.

   (3) Fixed Derivative Constraints 6 for start and the end.

2. Can you extend the previous question to the case in which the cost functional minimizes the r-th derivative and we have k segments?

   Still from Euler-Language equation, we have k(2r) unknown parameter

   (a) Waypoint constraint In this k segment will generate 2k constraint. As each segment will have a start constraint and an ending constraint.

   (b) Free Derivative Constraints: For k segment we have k-1 intermedia points. And each polynomial's order will be $N = 2r - 1$ So each point's constraints would be $N - 1 = 2r - 2$. Then the final number of this is $(k-1)(2r-2)$

   (c) Fixed Derivative Consraints

   $$2kr - 2k - 2(k-1)(r-1) = 2kr - 2k - 2kr + 2k + 2r - 2 = 2r - 2 \quad (18)$$

   This is the number of each constraints.

3 Team Work

This lab aims to enable the simulated quadrotor to complete a predefined drone racing course in the Unity simulator. The core implementation resides in

`trajectory_generation_pkg/src/trajectory_generation_node.cpp`, where we implemented the trajectory generation and tracking logic using the

`mav_trajectory_generation` library. This logic utilizes the position and orientation of the race gates as waypoints to produce smooth, high-order polynomial trajectories.

3.1 Implementation of WaypointFollower

3.1.1 Part 1.1: Current State Acquisition

Part 1.1 requires retrieving the current state of the UAV, specifically its world-frame position $\mathbf{x}$, from the ROS topic `/current_state`. This is handled in the `onCurrentState` callback function by using `tf::pointMsgToEigen` to convert the `geometry_msgs::Point` from the `nav_msgs::Odometry` message into an `Eigen::Vector3d` vector.

```
void onCurrentState(nav_msgs::Odometry const& cur_state) {
 // ...
 // Populate the variable x, which encodes the current world position of the
    UAV
 // ...
 tf::pointMsgToEigen(cur_state.pose.pose.position, x);
 // ...
}
```

3.1.2 Part 1.2: Optimized Trajectory Generation

Part 1.2 is the central component, responsible for generating an optimized trajectory for both position and yaw angle based on the gate poses received from the `/desired_traj_vertices` topic (`geometry_msgs::PoseArray`).

Positional Trajectory Generation ($D = 3$) We used the `mav_trajectory_generation` library for minimum-snap optimization of the position. Since the UAV's position $\mathbf{x} = [x, y, z]^T$ is 3-dimensional, the dimension D is set to 3.

The starting vertex (`start_position`) is constrained to the current UAV position $\mathbf{x}$ and its derivatives up to `SNAP` order (4th derivative) are set to zero using `makeStartOrEnd`. Intermediate waypoints (gate centers) only enforce the `POSITION` constraint. The final waypoint (`end_position`) is also constrained using `makeStartOrEnd` to ensure a smooth stop.

Yaw Trajectory Generation ($D = 1$) The yaw trajectory is optimized independently with dimension $D = 1$. Due to the 2π wrap-around problem, it is crucial to handle the yaw constraints with an **accumulated yaw** (`accumulated_yaw`). This ensures the trajectory is smooth even when passing through the $\pm\pi$ boundary. The difference between the current gate yaw and the accumulated yaw is normalized to the range $[-\pi, \pi]$ using $\text{atan2}(\sin(\cdot), \cos(\cdot))$, and then added to the accumulated value. This provides a continuous yaw reference.

The critical implementation snippets are shown below:

```
const int D = 3; // dimension of each vertex in the trajectory
// ... (Initialization)

// Start from the current position
start_position.makeStartOrEnd(x, SNAP);
vertices.push_back(start_position);
start_yaw.addConstraint(ORIENTATION, 0); // Start Yaw
yaw_vertices.push_back(start_yaw);

Eigen::Vector3d position;
double accumulated_yaw = 0;

for (auto i = 0; i < poseArray.poses.size()-1; ++i) {
  // Positional Vertices
  mav_trajectory_generation::Vertex middle_position(D);
  tf::pointMsgToEigen(poseArray.poses[i].position, position);
  middle_position.addConstraint(POSITION,position);
  vertices.push_back(middle_position);

  // Yaw Vertices: Handling angle wrap-around
  double current_wp_yaw = tf::getYaw(poseArray.poses[i].orientation);
  double yaw_diff = current_wp_yaw - accumulated_yaw;
  double normalized_diff = std::atan2(std::sin(yaw_diff), std::cos(yaw_diff));
  accumulated_yaw += normalized_diff;

  mav_trajectory_generation::Vertex middle_yaw(1);
  middle_yaw.addConstraint(ORIENTATION, accumulated_yaw);
  yaw_vertices.push_back(middle_yaw);
}

// End Position Vertex (makeStartOrEnd)
tf::pointMsgToEigen(poseArray.poses.back().position, position);
end_position.makeStartOrEnd(position, SNAP);
vertices.push_back(end_position);

// End Yaw Vertex (Constraint Orientation and Velocity)
// Similar wrap-around handling for the last waypoint
double end_wp_yaw = tf::getYaw(poseArray.poses.back().orientation);
double end_yaw_diff = end_wp_yaw - accumulated_yaw;
double normalized_end_diff = std::atan2(std::sin(end_yaw_diff), std::cos(
    end_yaw_diff));
accumulated_yaw += normalized_end_diff;

end_yaw.addConstraint(ORIENTATION, accumulated_yaw);
end_yaw.addConstraint(VELOCITY, 0.0); // Zero velocity at the end
yaw_vertices.push_back(end_yaw);

// ... (Segment Time Estimation and Optimization)
```

Segment times were estimated using `estimateSegmentTimes`, with maximum velocity

`v_max` set to 8.0 m/s and maximum acceleration `a_max` set to 4.0 m/s^2. The final trajectories were solved using the linear optimizer: `opt.solveLinear()`.

3.1.3 Part 1.3: Desired State Publishing

Part 1.3 implements trajectory tracking by publishing the desired state (`/desired_state`) to the controller via a timer running at 5 Hz.

The core logic involves calculating the elapsed time `sampling_time` since the trajectory started (`trajectoryStartTime`) and using this time to evaluate the optimized position (`trajectory`) and yaw (`yaw_trajectory`) curves.

- Sampling Time: `sampling_time` is the duration from `trajectoryStartTime` to the current time, capped at `trajectory.getMaxTime()`.
- Evaluation: `trajectory.evaluate(sampling_time, derivative_order)` is used to obtain the desired `POSITION`, `VELOCITY`, and `ACCELERATION`. The yaw angle is evaluated from `yaw_trajectory`.
- Message Population: The obtained desired position, linear velocity, and linear acceleration are packed into a `trajectory_msgs::MultiDOFJointTrajectoryPoint` message (`next_point`). The rotational component (quaternion) is generated from the desired yaw angle using `tf::createQuaternionMsgFromYaw`.

```
void publishDesiredState(ros::TimerEvent const& ev) {
 if (trajectory.empty()) return;

 trajectory_msgs::MultiDOFJointTrajectoryPoint next_point;
 ros::Duration time_passed = ros::Time::now() - trajectoryStartTime;
 next_point.time_from_start = ros::Duration(time_passed.toSec()); // Set
    time_from_start

 double sampling_time = time_passed.toSec(); // Use elapsed time
 if (sampling_time > trajectory.getMaxTime())
  sampling_time = trajectory.getMaxTime();

 // Evaluate Trajectories
 using namespace mav_trajectory_generation::derivative_order;
 Eigen::Vector3d des_position = trajectory.evaluate(sampling_time, POSITION);
 Eigen::Vector3d des_velocity = trajectory.evaluate(sampling_time, VELOCITY);
 Eigen::Vector3d des_accel = trajectory.evaluate(sampling_time, ACCELERATION);
 Eigen::VectorXd des_orientation =
   yaw_trajectory.evaluate(sampling_time, ORIENTATION);

 // Populate next_point
 geometry_msgs::Transform transform;
 tf::vectorEigenToMsg(des_position, transform.translation);
 transform.rotation = tf::createQuaternionMsgFromYaw(des_orientation(0));
 next_point.transforms.push_back(transform);

```

```
 geometry_msgs::Twist velocity;
 tf::vectorEigenToMsg(des_velocity, velocity.linear);
 next_point.velocities.push_back(velocity);

 geometry_msgs::Twist acceleration;
 tf::vectorEigenToMsg(des_accel, acceleration.linear);
 next_point.accelerations.push_back(acceleration);

 desiredStatePub.publish(next_point);
}
```

3.2 Race Course Performance and Submission Details

3.2.1 System Launch Sequence

The full trajectory generation and following system is launched via four separate terminal commands:

Terminal 1: Simulator `$ cd ~/vnav/tesse/lab4`
`$ ./lab4.x86_64`
Terminal 2: ROS Bridge `$ roslaunch tesse_ros_bridge tesse_quadrotor_bridge.launch.yaml`
Terminal 3: Controller and Trajectory Follower `$ roslaunch trajectory_generation traj_following.launch.py`
Terminal 4: Waypoint Publisher `$ roslaunch planner_pkg traj_gen.launch.py`

4 Discussion

This lab we connected the knowledge we learned before and successfully make the UAV fly through the trajectory we want. Despite the controller we also focus on the trajectory optimization. This is implemented by mav_trajectory_generation which the inner mathematical principle is constraint equations. With such constraints this forces the UAV fly a smooth trajectory.

5 Conclusions

Until now, we have known how to make a quadrotor fly. We know how to represent the state of the UAV, and we know how to transform through different coordinates. Later we define the trajectory and give out the flat output. With the dynamic model of the UAV, we can transform the wrench to the angular velocity of each rotor. Later refer to the paper we understand the geometry controller which give out how to compute the wrench from current state and desired state. After planner give out the points, we use trajectory optimization optimize a smooth trajectory. In this lab we can see how UAV fly through gates without hitting them.